

Navigation Encoding em Duas Etapas: de Máquinas de Turing Quânticas à Compilação Topology-Aware de Estruturas e Modelos Clássicos Treinados

José Yrikes de Oliveira Feitosa¹

¹Engenharia da Computação – Universidade Federal Rural de Pernambuco (UFRPE)
Unidade Acadêmica de Belo Jardim (UABJ) – Belo Jardim – PE – Brasil

jyrikes3@gmail.com

Abstract. *This article develops a two-stage mathematical formulation of Navigation Encoding. Starting from the physical semantics of finite-dimensional Hilbert spaces and the computational semantics of quantum Turing machines, we derive why classical functions must be embedded into reversible operators before coherent use. The addressed-memory unitary $U_D |a\rangle |b\rangle = |a\rangle |b \oplus D[a]\rangle$, already implemented in the `quantum-cq` library, is proved to be a particular case of a broader encoder-compiler. Stage one canonicalizes a typed heap, graph, or trained model into a hardware-independent navigation representation. Stage two realizes its operations as unitary circuits while jointly considering logical interactions, hardware coupling, routing, error, and optional controlled approximation. We provide proofs, circuits, examples, cost models, and a reversible compilation theorem for finite-precision classical inference models.*

Resumo. *Este artigo desenvolve uma formulação matemática em duas etapas para o Navigation Encoding. Partindo da semântica física dos espaços de Hilbert finito-dimensionais e da semântica computacional das máquinas de Turing quânticas, deduz-se por que funções clássicas devem ser embutidas em operadores reversíveis antes de seu uso coerente. A unidade de memória endereçada $U_D |a\rangle |b\rangle = |a\rangle |b \oplus D[a]\rangle$, já implementada na biblioteca `quantum-cq`, é demonstrada como caso particular de um encoder-compilador mais geral. A primeira etapa canonicaliza heap, grafo ou modelo treinado em uma representação independente de hardware. A segunda realiza suas operações como circuitos unitários, considerando interações lógicas, acoplamento físico, roteamento, erro e aproximações controladas. São apresentados provas, circuitos, exemplos, modelos de custo e um teorema de compilação reversível de inferência clássica em precisão finita.*

Palavras-chave: computação quântica; máquina de Turing quântica; encoding quântico; computação reversível; estruturas de dados; compilação quântica; modelos clássicos treinados.

1. Introdução Algoritmos clássicos recebem números, tensores, tabelas, árvores, grafos ou modelos treinados e manipulam esses objetos por instruções potencialmente irreversíveis. Um processador quântico, ao contrário, não executa diretamente operações arbitrárias sobre estruturas clássicas: ele evolui estados físicos segundo transformações lineares compatíveis com a mecânica quântica. Antes de uma informação clássica participar coerentemente de um algoritmo quântico, é necessário especificar não somente *onde* a informação será representada, mas principalmente *quais operações físicas* disponibilizarão essa informação ao restante do algoritmo.

Em Quantum Machine Learning (QML), o encoding é frequentemente apresentado como um mapa de características

$$\Phi : \mathbb{R}^d \longrightarrow \mathcal{H}_{2^n}, \quad x \longmapsto |\phi(x)\rangle = U_\Phi(x) |0\rangle^{\otimes n}, \quad (1)$$

no qual um vetor clássico parametriza a preparação de um estado. Essa formulação é essencial para circuitos variacionais e kernels quânticos Biamonte et al. (2017); Schuld and Killoran (2019); Havlíček et al. (2019), mas não esgota o conceito de encoding. Oráculos, Hamiltonianos, block-encodings, memórias endereçadas e operadores de navegação representam informação por meio da ação de uma transformação, e não apenas pela preparação isolada de um vetor.

A contribuição inicial do projeto quantum-cq foi a implementação e validação de uma memória endereçada finita

$$U_D |a\rangle |b\rangle = |a\rangle |b \oplus D[a]\rangle, \quad (2)$$

capaz de consultar uma tabela clássica em superposição, desfazer a consulta por *uncompute* e integrar oráculos derivados e busca de Grover Feitosa (2026a). O presente trabalho reposiciona essa unidade como o primeiro caso particular de uma tese mais ampla: o **Navigation Encoding em duas etapas**.

A hipótese central é que o encoder deve ser entendido como um compilador semântico:

$$S \xrightarrow{\text{StructEnc}} N_S \xrightarrow{\text{Realize}_G} (U_{S,G}, C_{S,G}) \quad (3)$$

A primeira etapa recebe uma estrutura clássica S e produz uma representação de navegação N_S , independente do backend. A segunda recebe N_S , uma operação requerida e um grafo físico G , produzindo um operador lógico e um circuito compatível com o hardware. A semântica deve ser preservada exatamente ou dentro de um erro explicitamente limitado.

Essa formulação permite tratar de maneira unificada:

- tabelas e memórias endereçadas;
- listas, árvores, heaps e grafos tipados;
- operações como `read`, `next`, `parent`, `child`, `neighbor` e `compare`;
- escolha conjunta de representação, placement, routing e síntese;
- compilação da função de inferência de modelos clássicos treinados para oráculos quânticos reversíveis.

O texto é simultaneamente dedutivo e construtivo. Primeiro, a física e as máquinas de Turing quânticas justificam a necessidade de unitariedade. Em seguida, funções clássicas são incorporadas em oráculos reversíveis. A memória U_D aparece como primeira realização concreta. Depois, formaliza-se a generalização estrutural, a realização topology-aware e a transferência de modelos treinados. O resultado final não afirma que qualquer modelo clássico se tornará um circuito eficiente, nem que a representação implica vantagem quântica. A afirmação demonstrada é mais precisa: sob domínios finitos, operações computáveis e primitivas reversíveis, existe uma compilação semanticamente correta para uma unidade quântica; sua eficiência permanece um problema explícito de síntese e arquitetura.

2. Da física quântica ao modelo computacional

2.1. Estados, composição e medição Um sistema quântico puro é descrito por um raio em um espaço de Hilbert complexo. Para um qubit,

$$\mathcal{H}_1 = \mathbb{C}^2, \quad |\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (4)$$

Um registrador de n qubits possui espaço

$$\mathcal{H}_n = (\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}. \quad (5)$$

O produto tensorial representa composição física. Estados separáveis têm a forma $|\psi_A\rangle \otimes |\psi_B\rangle$; estados gerais podem conter emaranhamento e não admitir tal fatoração.

Uma medição projetiva com projetores $\{P_m\}_m$ satisfaz

$$P_m P_{m'} = \delta_{m,m'} P_m, \quad \sum_m P_m = \mathbb{I}, \quad \Pr(m) = \langle \psi | P_m | \psi \rangle. \quad (6)$$

Para estados mistos, usa-se uma matriz densidade $\rho \succeq 0$, $\text{Tr } \rho = 1$, e $\Pr(m) = \text{Tr}(P_m \rho)$. Canais abertos são mapas completamente positivos e preservadores do traço. Entretanto, a parte coerente de um algoritmo de circuitos é modelada por operadores unitários sobre um sistema suficientemente ampliado Nielsen and Chuang (2010); Watrous (2018).

2.2. Por que a evolução fechada é unitária Se um sistema fechado possui Hamiltoniano hermitiano $H = H^\dagger$, a equação de Schrödinger

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = H |\psi(t)\rangle \quad (7)$$

possui solução

$$|\psi(t)\rangle = U(t) |\psi(0)\rangle, \quad U(t) = e^{-iHt/\hbar}. \quad (8)$$

Como H é hermitiano,

$$U(t)^\dagger U(t) = e^{iHt/\hbar} e^{-iHt/\hbar} = \mathbb{I}. \quad (9)$$

Logo, normas e produtos internos são preservados:

$$\langle U\psi | U\phi \rangle = \langle \psi | \phi \rangle. \quad (10)$$

Isso impede que dois estados ortogonais distintos sejam colapsados deterministicamente no mesmo estado durante a evolução coerente. A irreversibilidade observada em medição, descarte de subsistemas e ruído é descrita por canais, não por uma atualização fechada arbitrária.

A tese de Navigation Encoding usa essa distinção da seguinte forma: a consulta e as operações internas devem ser unidades, isometrias dilatadas ou canais com ambiente explicitado. O núcleo desenvolvido neste artigo trabalha com unidades, pois elas podem ser compostas, invertidas e usadas dentro de interferência e amplificação de amplitude.

2.3. Álgebras de operadores como linguagem do encoder O conjunto dos operadores lineares em $\mathcal{H}$ é denotado por $\mathcal{B}(\mathcal{H})$. Em dimensão finita,

$$\mathcal{B}(\mathbb{C}^d) \cong M_d(\mathbb{C}). \quad (11)$$

Com soma, composição, adjunto e norma operatorial, $\mathcal{B}(\mathcal{H})$ é uma C^* -álgebra. Um elemento u é unitário quando

$$u^*u = uu^* = \mathbb{I}. \quad (12)$$

Isso fornece uma mudança conceitual importante. O encoder não precisa ser somente um mapa $x \mapsto |\phi(x)\rangle$; pode ser um mapa

$$S \mapsto U_S \in \mathcal{U}(\mathcal{B}(\mathcal{H})), \quad (13)$$

no qual uma estrutura clássica parametriza um operador disponível ao algoritmo. Álgebras de operadores e C^* -álgebras finito-dimensionais são, portanto, a camada matemática natural para a tese Bény and Richter (2015); Landsman (1998); Feitosa (2026b).

3. Máquinas de Turing quânticas e o papel da unitariedade na tese

3.1. Configurações clássicas como base ortonormal Uma máquina de Turing clássica possui um conjunto finito de estados internos Q , um alfabeto de fita Γ , uma cabeça de leitura e uma função de transição. Uma configuração instantânea registra, por exemplo,

$$C = (q, i, T), \quad (14)$$

onde $q \in Q$ é o estado interno, $i \in \mathbb{Z}$ é a posição da cabeça e $T : \mathbb{Z} \rightarrow \Gamma$ é o conteúdo da fita com suporte finito fora do símbolo branco.

Na máquina de Turing quântica (MTQ), o conjunto contável de configurações clássicas $\mathcal{C}$ indexa uma base ortonormal de

$$\mathcal{H}_{\text{conf}} = \ell^2(\mathcal{C}) = \left\{ \psi : \mathcal{C} \rightarrow \mathbb{C} \mid \sum_{C \in \mathcal{C}} |\psi(C)|^2 < \infty \right\}. \quad (15)$$

O vetor $|C\rangle$ representa uma configuração clássica definida. Um estado geral da máquina é

$$|\Psi\rangle = \sum_{C \in \mathcal{C}} \alpha_C |C\rangle, \quad \sum_C |\alpha_C|^2 = 1. \quad (16)$$

Deutsch introduziu esse modelo como uma formulação universal de computação coerente, posteriormente desenvolvida em teoria de complexidade por Bernstein e Vazirani Deutsch (1985); Bernstein and Vazirani (1997).

3.2. Transição local e operador global Uma função de transição quântica local atribui amplitudes a mudanças de estado, escrita na fita e movimento da cabeça. Em notação simplificada,

$$\delta(q, \sigma; q', \tau, d) \in \mathbb{C}, \quad d \in \{-1, +1\}. \quad (17)$$

Ela induz um operador global U_δ por sua ação nos vetores-base:

$$U_\delta |C\rangle = \sum_{C' \in \mathcal{C}} \alpha_{C,C'} |C'\rangle. \quad (18)$$

A extensão para superposições é determinada por linearidade:

$$U_\delta \left(\sum_C \beta_C |C\rangle \right) = \sum_C \beta_C U_\delta |C\rangle. \quad (19)$$

As condições locais de boa formação devem garantir que U_δ seja unitário. Globalmente, isso equivale à ortonormalidade das colunas da matriz de transição:

$$\sum_{C'} \alpha_{C,C'}^* \alpha_{D,C'} = \delta_{C,D} \quad \text{para todos } C, D \in \mathcal{C}. \quad (20)$$

Em palavras, distribuições de amplitude geradas por configurações ortogonais continuam ortogonais. Caracterizações locais completas precisam ainda impedir interferências inconsistentes entre transições cujas cabeças alcançam a mesma configuração; esse problema foi formalizado na literatura de funções de transição locais de MTQs Bernstein and Vazirani (1997); Ozawa and Nishimura (2000).

Proposição 3.1 (Conservação probabilística da MTQ). *Se U_δ satisfaz a Equação (20) e é sobrejetivo, então U_δ é unitário e, para todo estado normalizado $|\Psi\rangle$,*

$$\|U_\delta |\Psi\rangle\|_2 = 1. \quad (21)$$

Demonstração. A Equação (20) implica $U_\delta^\dagger U_\delta = \mathbb{I}$, isto é, U_δ é uma isometria. A sobrejetividade fornece $U_\delta U_\delta^\dagger = \mathbb{I}$. Logo, U_δ é unitário e preserva a norma por (10). $\square$

3.3. Equivalência com circuitos e consequência para encoders O modelo de circuitos descreve a computação como um produto de unidades locais

$$U = U_T U_{T-1} \cdots U_2 U_1. \quad (22)$$

Yao demonstrou equivalência polinomial entre famílias uniformes de circuitos quânticos e MTQs eficientes Yao (1993). Portanto, a exigência de que uma etapa coerente seja unitária não é uma convenção específica de uma biblioteca: ela deriva do modelo computacional e de sua semântica física.

Essa ponte produz a regra fundamental da tese:

Uma representação clássica só pode participar internamente de uma computação quântica fechada quando suas operações observáveis são realizadas por unidades, isometrias estendidas ou canais explicitamente dilatados.

Consequentemente, não basta dizer que uma heap, grafo ou modelo “foi colocado em qubits”. É necessário definir a transformação física que implementa **read**, **follow**, **compare** ou **infer**, provar sua correção e medir o custo de sua decomposição.

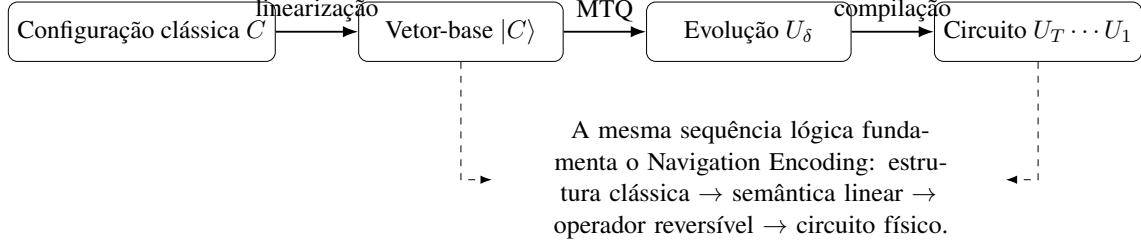


Figura 1. Da configuração clássica à realização circuital.

4. De funções clássicas a oráculos reversíveis

4.1. Por que o mapa ingênuo falha

$$f : \{0, 1\}^n \longrightarrow \{0, 1\}^m. \quad (23)$$

A transformação ingênua

$$|x\rangle \longmapsto |f(x)\rangle \quad (24)$$

só define uma unidade entre espaços de mesma dimensão quando f é bijetiva. Se $x_1 \neq x_2$ e $f(x_1) = f(x_2)$, então

$$\langle x_1 | x_2 \rangle = 0, \quad \langle f(x_1) | f(x_2) \rangle = 1, \quad (25)$$

violando a preservação do produto interno.

O problema não é a função clássica ser não invertível por si só. O problema é apagar a informação necessária para inverter a transformação completa.

4.2. Embedding XORA construção padrão preserva a entrada e acumula a saída em um segundo registrador:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle. \quad (26)$$

Teorema 4.1 (Embedding reversível de função finita). *Para toda função $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$, a extensão linear de U_f definida em (26) é unitária e involutiva:*

$$U_f^\dagger = U_f, \quad U_f^2 = \mathbb{I}. \quad (27)$$

Demonstração. Para cada par (x, y) , o mapa

$$(x, y) \mapsto (x, y \oplus f(x)) \quad (28)$$

é uma permutação da base computacional. Aplicando-o duas vezes,

$$y \oplus f(x) \oplus f(x) = y. \quad (29)$$

Logo, $U_f^2 = \mathbb{I}$. Uma permutação de uma base ortonormal é unitária, e sua inversa coincide com ela própria. $\square$

Essa construção liga diretamente computação reversível, oráculos quânticos e Navigation Encoding. Bennett mostrou que computações clássicas podem ser simuladas reversivelmente preservando e posteriormente limpando informação intermediária Bennett (1973). O custo adicional de ancillas, cópias e recomputação não desaparece; ele se torna parte explícita da complexidade.

4.3. Compute–copy–uncompute Uma computação reversível intermediária pode produzir lixo $g(x)$:

$$A_f |x\rangle |0\rangle |0\rangle = |x\rangle |f(x)\rangle |g(x)\rangle. \quad (30)$$

Para obter um oráculo limpo, adiciona-se um registrador de saída, copia-se $f(x)$ por XOR e aplica-se $A_f^\dagger$:

$$\begin{aligned} |x\rangle |0\rangle |0\rangle |0\rangle &\xrightarrow{A_f} |x\rangle |f(x)\rangle |g(x)\rangle |0\rangle \\ &\xrightarrow{\text{COPY}} |x\rangle |f(x)\rangle |g(x)\rangle |f(x)\rangle \\ &\xrightarrow{A_f^\dagger} |x\rangle |0\rangle |0\rangle |f(x)\rangle. \end{aligned} \quad (31)$$

O procedimento será reutilizado na compilação de modelos clássicos treinados.

5. Primeira implementação: memória endereçada como caso particular

5.1. Definição e forma operatorial

$$A = \{0, 1\}^n, \quad B = \{0, 1\}^w, \quad D : A \rightarrow B. \quad (32)$$

A função D representa uma memória clássica de 2^n endereços e palavras de w bits. Defina os espaços

$$\mathcal{H}_A = \text{span}\{|a\rangle : a \in A\}, \quad \mathcal{H}_B = \text{span}\{|b\rangle : b \in B\}. \quad (33)$$

Para $z \in B$, defina a translação XOR

$$X_z |b\rangle = |b \oplus z\rangle. \quad (34)$$

O operador de memória é

$$U_D = \sum_{a \in A} |a\rangle\langle a| \otimes X_{D[a]}. \quad (35)$$

Sua ação é exatamente

$$U_D |a\rangle |b\rangle = |a\rangle |b \oplus D[a]\rangle. \quad (36)$$

Proposição 5.1 (Unitariedade e involução de U_D). *O operador U_D é unitário, hermitiano e involutivo:*

$$U_D^\dagger = U_D, \quad U_D^2 = \mathbb{I}_{A \otimes B}. \quad (37)$$

Demonstração. Cada $X_{D[a]}$ é uma permutação e satisfaz $X_{D[a]}^2 = \mathbb{I}_B$. Os projetores $P_a = |a\rangle\langle a|$ são ortogonais e somam a identidade. Portanto,

$$U_D^2 = \sum_{a, a'} P_a P_{a'} \otimes X_{D[a]} X_{D[a']} \quad (38)$$

$$= \sum_a P_a \otimes X_{D[a]}^2 \quad (39)$$

$$= \sum_a P_a \otimes \mathbb{I}_B = \mathbb{I}_A \otimes \mathbb{I}_B. \quad (40)$$

Como $U_D^{-1} = U_D$ e U_D é real na base computacional, $U_D^\dagger = U_D$. □

5.2. Consulta em superposição

$$|\psi_A\rangle = \sum_{a \in A} \alpha_a |a\rangle, \quad \sum_a |\alpha_a|^2 = 1, \quad (41)$$

então, por linearidade,

$$U_D \left(\sum_a \alpha_a |a\rangle |0^w\rangle \right) = \sum_a \alpha_a |a\rangle |D[a]\rangle. \quad (42)$$

A Equação (42) não permite ler todos os valores por uma única medição. Ela disponibiliza todos os pares endereço–dado coerentemente para interferência, avaliação de predicados, phase kickback e uncompute. Essa é uma semântica de consulta QRAM-like circuital, sem afirmar a existência de uma QRAM física escalável Giovannetti et al. (2008b,a).

5.3. Síntese lógica por controles de padrão Escreva $D[a]_j$ para o bit j da palavra armazenada. Para cada par (a, j) tal que $D[a]_j = 1$, aplica-se uma porta X no qubit de dados b_j , controlada pelo padrão completo do endereço. Controles em zero podem ser implementados por conjugação com X .

Se $a = (a_{n-1}, \dots, a_0)$, a porta elementar é

$$G_{a,j} = \left(\prod_{k:a_k=0} X_{A_k} \right) \text{MCX}(A_{n-1}, \dots, A_0; B_j) \left(\prod_{k:a_k=0} X_{A_k} \right). \quad (43)$$

Então

$$U_D = \prod_{a \in A} \prod_{j:D[a]_j=1} G_{a,j}. \quad (44)$$

As portas podem ser ordenadas livremente nesse caso, pois controles associados a endereços distintos têm suportes ortogonais, e traduções XOR no mesmo endereço comutam.

O número lógico de escritas controladas é

$$N_{\text{write}}(D) = \sum_{a \in A} h(D[a]), \quad (45)$$

onde $h(z)$ é o peso de Hamming. Esse custo deve ser separado de:

- custo após decomposição de MCX em portas nativas;
- custo de routing imposto pelo grafo de acoplamento;
- custo transpilado após otimizações e restrições físicas.

5.4. Exemplo completo com quatro endereços

$$D[00] = 01, \quad D[01] = 10, \quad D[10] = 11, \quad D[11] = 00. \quad (46)$$

O peso total é $1 + 1 + 2 + 0 = 4$. O circuito lógico contém quatro escritas controladas, mostrado na Figura 2. Controles vazios significam condição no estado $|0\rangle$.

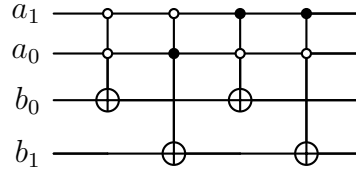


Figura 2. Circuito lógico de U_D para $D = [01, 10, 11, 00]$.

A ação sobre a base é:

Entrada	Operação no dado	Saída
$ 00\rangle 00\rangle$	$00 \oplus 01$	$ 00\rangle 01\rangle$
$ 01\rangle 00\rangle$	$00 \oplus 10$	$ 01\rangle 10\rangle$
$ 10\rangle 00\rangle$	$00 \oplus 11$	$ 10\rangle 11\rangle$
$ 11\rangle 00\rangle$	$00 \oplus 00$	$ 11\rangle 00\rangle$

Com endereço uniforme,

$$|\psi_0\rangle = \frac{1}{2} \sum_{a \in \{00,01,10,11\}} |a\rangle |00\rangle, \quad (47)$$

obtemos

$$U_D |\psi_0\rangle = \frac{1}{2} (|00\rangle |01\rangle + |01\rangle |10\rangle + |10\rangle |11\rangle + |11\rangle |00\rangle). \quad (48)$$

5.5. Query-predicate-unquery A consulta torna-se algoritmicamente útil quando é temporária. Para um predicado $\chi : B \rightarrow \{0, 1\}$, constrói-se um marcador de fase

$$O_\chi |b\rangle = (-1)^{\chi(b)} |b\rangle. \quad (49)$$

O oráculo sobre endereços é

$$O_{D,\chi} = U_D (\mathbb{I}_A \otimes O_\chi) U_D. \quad (50)$$

Aplicado a $|a\rangle |0\rangle$,

$$O_{D,\chi} |a\rangle |0\rangle = (-1)^{\chi(D[a])} |a\rangle |0\rangle. \quad (51)$$

O dado é limpo, mas a fase permanece no endereço. A Figura 3 mostra o padrão.

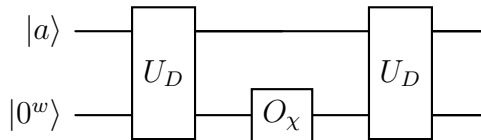


Figura 3. Fluxo query-predicate-unquery.

Se $\chi(b) = 1$ somente para $b = 11$ no exemplo, apenas o endereço 10 recebe fase negativa. O oráculo pode ser usado pelo operador de difusão de Grover (1996).

5.6. Estado experimental da primeira implementação A biblioteca quantum-cq validou semanticamente o operador, sua involução, consulta em superposição, readback, uncompute, oráculos derivados e Grover em simulação ideal Feitosa (2026a). Para a memória experimental $D = [3, 5, 7, 9]$, três engines produziram a mesma estrutura atual de custo:

Tabela 1. Custo observado para $D = [3, 5, 7, 9]$ na implementação inicial.

Engine	Qubits	Prof.	Size	CCX	X	CX
explicit	6	22	27	9	18	54
sparse	6	22	27	9	18	54
gram-like	6	22	27	9	18	54

O resultado mostra que os nomes das engines ainda não representam otimizações circuitais distintas para esse caso. A contribuição comprovada é semântica e estrutural; uma vantagem de custo precisa ser demonstrada por novas realizações. Testes de transpilation sem restrição topológica foram executados, enquanto mapas restritos linear, anel e completo apresentaram falhas de configuração no experimento de fechamento. Portanto, a generalização topology-aware a seguir é uma formulação e agenda de implementação, não um resultado físico já confirmado.

6. Limite da memória plana e introdução da navegação estrutural O operador U_D assume que a operação natural é consultar uma palavra por endereço absoluto. Em listas, árvores e grafos, a operação de interesse pode ser “seguir o próximo nó”, “retornar ao pai”, “escolher o filho j ” ou “obter o j -ésimo vizinho”. O índice físico de um nó não é a própria semântica de ponteiro.

Considere uma função de resolução

$$\rho_S : P \times F \longrightarrow P \sqcup V \sqcup \{\perp\}, \quad (52)$$

na qual:

- P é o conjunto de ponteiros lógicos ou identidades de nós;
- F é o conjunto de campos, seletores ou deslocamentos;
- V é o domínio de valores;
- $\perp$ indica referência inexistente ou acesso inválido.

A memória plana é recuperada escolhendo $P = A$, um único campo `value` e $\rho_S(a, \text{value}) = D[a]$.

A separação entre ponteiro lógico e qubit físico é essencial. Um ponteiro pertence à semântica da estrutura; sua codificação binária, qudit, setor ou região física pertence à realização.

7. Primeira etapa: encoding estrutural-semântico

7.1. Estruturas tipadas e equivalência por renomeação

Definição 7.1 (Estrutura de navegação finita). *Uma estrutura de navegação finita é uma tupla*

$$S = (P, T, \tau, F, \rho, r, \mu, \mathcal{I}), \quad (53)$$

na qual P é o conjunto finito de nós; T é o conjunto de tipos; $\tau : P \rightarrow T$ atribui um tipo; F_t é o conjunto de campos admissíveis para o tipo t ; ρ resolve cada par nó–campo; $r \in P$ é uma raiz opcional; μ é uma distribuição ou perfil de acessos; e $\mathcal{I}$ é um conjunto de invariantes estruturais.

Dois objetos podem usar endereços diferentes e ainda representar a mesma estrutura. Defina $S \cong S'$ quando existe uma bijeção $h : P \rightarrow P'$ que preserva raiz, tipos, valores, campos e relações:

$$h(\rho_S(p, f)) = \rho_{S'}(h(p), f) \quad (54)$$

para referências a nós, com preservação análoga para valores e $\perp$. O domínio semântico correto da primeira etapa é o quociente $\mathfrak{S}/\cong$, não a representação nominal crua.

Uma função de canonicalização

$$\text{can} : \mathfrak{S}/\cong \longrightarrow \mathfrak{S}_{\text{can}} \quad (55)$$

escolhe um representante determinístico, por exemplo por travessia a partir da raiz, ordenação estável de campos e desempate por tipos e valores. Para grafos gerais, canonicalização eficiente pode ser difícil; a tese exige que a política adotada seja computável e que equivalências não resolvidas sejam tratadas explicitamente, sem assumir um algoritmo universal barato.

7.2. Espaço estrutural abstrato e realização finita Como semântica denotacional, classes de estruturas podem indexar uma base formal

$$\mathcal{H}_{\text{struct}} = \ell^2(\mathfrak{S}/\cong), \quad |[S]\rangle \in \mathcal{H}_{\text{struct}}. \quad (56)$$

Essa construção permite falar matematicamente de combinações lineares de estruturas. Contudo, ela não obriga a implementação a preparar uma superposição de heaps inteiras. Na tese implementável, S é normalmente um objeto clássico fixo que parametriza operadores sobre registradores finitos. Cada backend utiliza somente um setor finito, com um número determinado de qubits.

Setores de tamanhos diferentes podem ser organizados por soma direta

$$\mathcal{H}_{\leq N} = \bigoplus_{k=0}^N \mathcal{H}_k. \quad (57)$$

Espaços de Fock seriam apropriados se criação e remoção de um número variável de componentes fossem operações físicas centrais; não são necessários para a primeira realização de memória e navegação finitas Feitosa (2026b).

7.3. Representação intermediária de navegação A primeira etapa é definida por

$$\text{StructEnc} : \mathfrak{S} / \cong \longrightarrow \mathfrak{N}, \quad [S] \longmapsto N_S, \quad (58)$$

com

$$N_S = (P, F, R, \bar{\rho}, \Omega, \mu, \mathcal{I}, \mathcal{T}). \quad (59)$$

Aqui $R = P \sqcup V \sqcup \{\perp\}$ é o domínio tagged de resultados, $\bar{\rho}$ é a versão totalizada da resolução, Ω é o conjunto de operações requeridas e $\mathcal{T}$ registra tipos e larguras.

A totalização é necessária porque uma unidade deve estar definida em toda a base física. Acessos inválidos podem retornar um código reservado $\text{enc}(\perp)$, sem colapsar estados ou lançar uma exceção clássica no meio do circuito.

7.4. Álgebra de navegação O dado estruturado disponibiliza operações naturais. Seja $\mathcal{A}_{\text{nav}}(S)$ a $*$ -álgebra gerada por símbolos correspondentes a:

$$\Omega = \{\text{read}, \text{follow}_f, \text{parent}, \text{child}_j, \text{neighbor}_j, \text{compare}_x, \dots\}, \quad (60)$$

com relações impostas pelos invariantes de S . A segunda etapa construirá uma representação

$$\pi_{S,G} : \mathcal{A}_{\text{nav}}(S) \longrightarrow \mathcal{B}(\mathcal{H}_G). \quad (61)$$

Quando os geradores são realizados por unidades finitas, a álgebra matricial por eles gerada e sua aderência em norma formam uma C^* -subálgebra de $\mathcal{B}(\mathcal{H}_G)$. Assim, a tese não substitui Hilbert: Hilbert fornece a semântica física, enquanto a álgebra organiza as operações estruturais.

Entrada: Estrutura tipada S , operações Ω , perfil de acesso μ

Saída: IR de navegação N_S

Validar tipos, referências e invariantes $\mathcal{I}$

Calcular ou registrar a classe de equivalência por renomeação

Canonicalizar identidades quando possível

Separar ponteiros, campos, valores e tags de resultado

Totalizar acessos inválidos com $\perp$

Anotar frequências de acesso e relações lógicas

Emitir $N_S = (P, F, R, \bar{\rho}, \Omega, \mu, \mathcal{I}, \mathcal{T})$

Algorithm 1: Etapa 1: StructEnc.

8. Segunda etapa: realização quântica reversível

8.1. Codificações finitas

Escolham-se injeções

$$e_P : P \hookrightarrow \{0, 1\}^{n_P}, \quad (62)$$

$$e_F : F \hookrightarrow \{0, 1\}^{n_F}, \quad (63)$$

$$e_R : R \hookrightarrow \{0, 1\}^{w_R}. \quad (64)$$

As imagens não utilizadas são estados de padding e devem receber uma semântica total definida. A realização básica da resolução é

$$U_{\rho,S} |e_P(p)\rangle |e_F(f)\rangle |b\rangle = |e_P(p)\rangle |e_F(f)\rangle |b \oplus e_R(\bar{\rho}(p, f))\rangle. \quad (65)$$

Em forma operatorial,

$$U_{\rho,S} = \sum_{p,f} |e_P(p), e_F(f)\rangle \langle e_P(p), e_F(f)| \otimes X_{e_R(\bar{\rho}(p,f))} + U_{\text{pad}}, \quad (66)$$

onde U_{pad} especifica uma unidade nos estados fora das imagens válidas, por exemplo identidade.

Teorema 8.1 (Unitariedade da navegação totalizada). *Se e_P , e_F e e_R são injetivas e U_{pad} é unitário em um subespaço ortogonal ao domínio válido, então $U_{\rho,S}$ é unitário. No domínio válido, $U_{\rho,S}^2 = \mathbb{I}$.*

Demonstração. Cada par válido (p, f) seleciona, por um projetor ortogonal, uma translação XOR unitária no registrador de resultado. A soma direta de blocos unitários em subespaços ortogonais é unitária. Nos blocos válidos, a tradução é involutiva. O bloco de padding é unitário por hipótese. $\square$

8.2. Correção semântica Seja $\mathcal{H}_S^{\text{abs}}$ o espaço lógico abstrato das configurações de navegação e seja

$$V_{S,G} : \mathcal{H}_S^{\text{abs}} \hookrightarrow \mathcal{H}_G \quad (67)$$

uma isometria que codifica estados lógicos em qubits físicos. Para uma operação $o \in \Omega$, denote por U_o^{abs} seu embedding reversível abstrato e por $C_{S,o,G}$ o circuito produzido.

Definição 8.2 (Preservação semântica exata). *A realização é exata se*

$$\llbracket C_{S,o,G} \rrbracket V_{S,G} = V_{S,G} U_o^{\text{abs}}. \quad (68)$$

A condição forma um diagrama comutativo:

$$\begin{array}{ccc} \mathcal{H}_S^{\text{abs}} & \xrightarrow{U_o^{\text{abs}}} & \mathcal{H}_S^{\text{abs}} \\ V_{S,G} \downarrow & & \downarrow V_{S,G} \\ \mathcal{H}_G & \xrightarrow{\llbracket C_{S,o,G} \rrbracket} & \mathcal{H}_G \end{array}$$

Figura 4. Comutatividade exigida entre semântica abstrata e realização física.

Para uma realização aproximada,

$$\|\llbracket C_{S,o,G} \rrbracket V_{S,G} - V_{S,G} U_o^{\text{abs}}\|_{\text{op}} \leq \varepsilon_{\text{sem}}. \quad (69)$$

Quando ruído e descarte são modelados como canais, a norma diamante é mais apropriada.

8.3. Definição completa do encoder em duas etapas

Definição 8.3 (Navigation Encoding em duas etapas). *Dado um objeto estrutural S , um conjunto de operações Ω e um backend descrito por G , define-se*

$$\text{NavEnc}_G(S, \Omega) = \text{Realize}_G(\text{StructEnc}([S]), \Omega) \quad (70)$$

$$= (\{U_{S,o}\}_{o \in \Omega}, \{C_{S,o,G}\}_{o \in \Omega}, \Pi), \quad (71)$$

onde Π contém provas ou certificados de correção, metadados de custo e limites de erro.

A memória U_D é recuperada escolhendo uma estrutura com um campo de valor e a operação `read`. Portanto, ela não é substituída; torna-se a primitiva mínima da generalização.

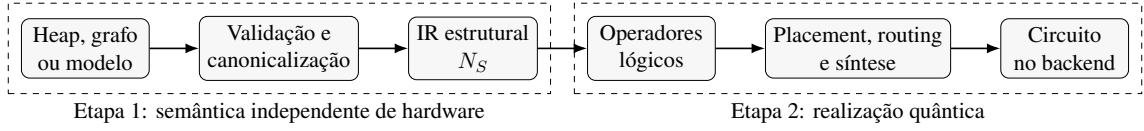


Figura 5. Arquitetura do encoder-compilador em duas etapas.

9. Realização topology-aware

9.1. Do circuito lógico ao grafo de interação O termo *topology-aware* é usado aqui no sentido de topologia de conectividade do hardware, não de fases topológicas da matéria. Um circuito lógico induz um grafo de interação

$$G_L = (Q_L, E_L, w), \quad (72)$$

no qual os vértices são qubits ou blocos lógicos e w_{uv} mede frequência, criticidade ou custo das interações entre u e v . Para o Navigation Encoding, qubits de endereço frequentemente controlam muitas operações em qubits de dados; eles tendem a ser vértices de alta centralidade em G_L .

O processador é modelado por

$$G_H = (Q_H, E_H, \epsilon, t, T_1, T_2), \quad (73)$$

com arestas de interação nativa, erros ϵ_e , durações t_e e parâmetros de coerência. Um placement é uma injeção

$$\pi : Q_L \hookrightarrow Q_H. \quad (74)$$

Um custo de distância básico é

$$C_{\text{dist}}(\pi) = \sum_{\{u,v\} \in E_L} w_{uv} d_H(\pi(u), \pi(v)), \quad (75)$$

onde d_H é a menor distância no grafo físico. Uma versão noise-aware pode usar

$$C_{\text{phys}}(\pi) = \sum_{\{u,v\} \in E_L} w_{uv} \left[\alpha d_H + \beta (-\log F_{\text{path}}) + \gamma \frac{t_{\text{path}}}{T_2^{\text{eff}}} \right]. \quad (76)$$

9.2. Exemplo simples de placement Considere três qubits lógicos: a_0 interage dez vezes com b_0 e oito vezes com b_1 , enquanto b_0 e b_1 não interagem. Em uma linha física $q_0 - q_1 - q_2$:

- placement central: $\pi(a_0) = q_1, \pi(b_0) = q_0, \pi(b_1) = q_2$;
- placement extremo: $\pi(a_0) = q_0, \pi(b_0) = q_1, \pi(b_1) = q_2$.

Os custos são

$$C_{\text{central}} = 10 \cdot 1 + 8 \cdot 1 = 18, \quad (77)$$

$$C_{\text{extremo}} = 10 \cdot 1 + 8 \cdot 2 = 26. \quad (78)$$

A semântica lógica é a mesma; a realização física é potencialmente mais barata no primeiro caso.

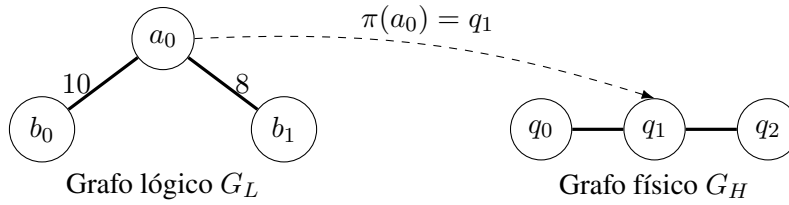


Figura 6. Placement orientado pelo grafo de interação.

9.3. Routing, permutações e correção Se uma porta de dois qubits exige uma aresta ausente, o roteador insere SWAPs. Seja P_π a permutação que identifica a ordem lógica com a física. A correção ideal exige

$$\llbracket C_{\text{routed}} \rrbracket = P_{\pi_{\text{out}}} \llbracket C_{\text{logical}} \rrbracket P_{\pi_{\text{in}}}^\dagger, \quad (79)$$

com atualização explícita do mapeamento se o placement final difere do inicial. Algoritmos como alocação por grafo, SABRE e técnicas de routing estudam esse problema NP-difícil por heurísticas Siraichi et al. (2018); Li et al. (2019); Cowtan et al. (2019).

A novidade proposta é antecipar informações da estrutura antes de fixar a IR física. Em um pipeline *encode-first-route-later*, endereços e campos são escolhidos sem conhecer o hardware. No pipeline conjunto, a codificação de ponteiros, agrupamento de campos e placement podem ser escolhidos cooperativamente, desde que a Equação (68) permaneça válida.

9.4. Otimização multiobjetivo Seja θ o conjunto de decisões: canonicalização, codificação binária, decomposição de operadores, ancillas, placement π , routing r e síntese s . O encoder pode resolver

$$\min_{\theta} \lambda_g C_{\text{gates}}(\theta) + \lambda_r C_{\text{routing}}(\theta) + \lambda_n C_{\text{noise}}(\theta) + \lambda_a C_{\text{anc}}(\theta) + \lambda_s C_{\text{spectral}}(\theta), \quad (80)$$

sujeito a

$$\|\llbracket C_\theta \rrbracket V_S - V_S U_S\| \leq \varepsilon. \quad (81)$$

Para consulta exata, $\varepsilon = 0$. O custo de colisão semântica deve ser uma restrição dura: duas estruturas distinguíveis pelas operações requeridas não podem ser fundidas inadvertidamente.

Entrada: IR N_S , operação o , grafo físico G_H , tolerância ε
Saída: Unidade lógica $U_{S,o}$, circuito $C_{S,o,G}$, certificado Π
 Escolher codificações finitas injetivas e política de padding
 Construir o embedding reversível abstrato de o
 Gerar o grafo de interação lógico G_L
 Selecionar decomposição, ancillas e placement inicial
 Rotear interações não nativas e sintetizar portas do backend
 Verificar equivalência exata ou limite ε
 Coletar qubits, profundidade, portas de dois qubits, erro estimado e provenance
 Emitir $(U_{S,o}, C_{S,o,G}, \Pi)$

Algorithm 2: Etapa 2: Realize_G .

10. Colisões, entropia e estrutura espectral

10.1. Três noções de colisão É necessário distinguir:

1. **equivalência semântica:** $S \cong S'$ por renomeação de nós;
2. **equivalência observacional:** nenhuma operação de Ω distingue S de S' ;
3. **colisão de encoding:** representações iguais para estruturas que deveriam ser distinguíveis.

Uma colisão nominal é aceitável após quociente por isomorfismo. Uma colisão real viola a injetividade necessária ao embedding isométrico.

10.2. Entropia de acesso Se p_i é a probabilidade de acessar a posição i , a probabilidade de colisão de duas amostras independentes é

$$P_{\text{coll}} = \sum_i p_i^2. \quad (82)$$

A entropia de Rényi de ordem 2 é

$$H_2(P) = -\log_2 \sum_i p_i^2. \quad (83)$$

Para quatro endereços uniformes, $H_2 = 2$ bits. Para $(0.7, 0.1, 0.1, 0.1)$,

$$H_2 = -\log_2(0.52) \approx 0.943. \quad (84)$$

A concentração pode orientar placement de regiões quentes, cache lógico, ordenação de controles e escolha de representações. Ela não altera por si só a correção do operador.

10.3. Laplaciano e esparsificação Uma estrutura relacional induz um grafo lógico $G = (V, E)$, matriz de adjacência A e matriz de grau Δ . O Laplaciano é

$$L = \Delta - A. \quad (85)$$

Seus autovalores e autovetores descrevem conectividade, componentes, gargalos e modos de difusão Chung (1997). Um grafo $\tilde{G}$ é um esparsificador espectral de G se, para todo vetor x ,

$$(1 - \varepsilon)x^T Lx \leq x^T \tilde{L}x \leq (1 + \varepsilon)x^T Lx. \quad (86)$$

Tais aproximações são úteis para walks, difusão e energia quadrática Spielman and Srivastava (2011). Contudo, proximidade espectral não preserva automaticamente cada ponteiro ou lookup. Assim:

- consulta exata exige compressão semanticamente equivalente;
- realização aproximada deve declarar qual observável é preservado e qual norma limita o erro;
- uma mesma estrutura pode ser esparsa em arestas, bits armazenados ou coeficientes espectrais; essas propriedades não são equivalentes.

11. Camadas matemáticas adicionais e seu papel A formulação principal necessita apenas de espaços de Hilbert finitos, operadores, somas diretas, grafos e computação reversível. Outras estruturas enriquecem o projeto sem substituir esse núcleo:

Tabela 2. Estruturas matemáticas e prioridade para a tese.

Formalismo	Papel	Prioridade
Hilbert finito	Estados, tensor, adjunto e circuitos físicos	Essencial
Álgebras de operadores/ C^*	Operadores como objetos centrais; $S \mapsto U_S$	Essencial
Somas diretas	Setores de tamanho ou tipo diferentes	Alta
Matrizes	Ruído, mistura e backend aberto	Alta
densidade/canais		
ZX e categorias dagger	Reescrita e verificação diagramática de circuitos	Alta no compilador
Fibrados de Hilbert	Famílias de espaços/realizações parametrizadas	Futura
Espaço de Fock	Número variável de componentes físicos	Condicional
Álgebras de von Neumann	Sistemas infinitos e topologias fracas/fortes	Não necessária agora
Rigged Hilbert/Krein	Espectro contínuo ou métricas indefinidas	Fora do núcleo

O cálculo ZX pode atuar após a geração de unidades lógicas, simplificando circuitos por regras diagramáticas semanticamente corretas Duncan et al. (2020). Fibrados podem organizar famílias de espaços por backend ou parâmetro, mas não são requisito para implementar U_D ou $U_{\rho,S}$.

12. Resultado final: de um modelo clássico treinado a um oráculo quântico

12.1. O que significa transferir um modelo Um modelo clássico treinado não é simplesmente uma lista de pesos. Ele é uma função de inferência associada a um grafo computacional:

$$M = (G_M, \Theta, \mathcal{X}, \mathcal{Y}, f_\Theta), \quad (87)$$

com nós de operação, tensores, parâmetros e dependências. A transferência semanticamente forte não consiste em mapear cada peso para um ângulo arbitrário; consiste em produzir

uma unidade que realize a mesma função de inferência, após uma especificação finita de precisão.

O pipeline é

$$M_{\text{clássico}} \xrightarrow{\text{StructEnc}} N_M \xrightarrow{\text{Realize}_G} U_{M,p,G}. \quad (88)$$

A primeira etapa extrai o DAG, tipos, formas, parâmetros, quantização e operações. A segunda compila as primitivas para aritmética reversível, consultas de parâmetros e uncompute.

12.2. Quantização e domínio finito Circuitos finitos não representam reais com precisão infinita. Escolha um formato de ponto fixo com p bits e escala s . Uma quantização típica é

$$Q_{p,s}(x) = \text{clip}_{[q_{\min}, q_{\max}]}(\lfloor 2^s x \rfloor). \quad (89)$$

O modelo realizado é

$$\hat{f}_{\Theta,p} : X_p \rightarrow Y_p, \quad (90)$$

não a função real ideal sem discretização. Quantização inteira já é usada em inferência clássica eficiente Jacob et al. (2018); aqui ela também torna o domínio reversível finito.

A decomposição de erro pode ser organizada como

$$\varepsilon_{\text{total}} \leq \varepsilon_{\text{quant}} + \varepsilon_{\text{arith}} + \varepsilon_{\text{synth}} + \varepsilon_{\text{noise}}. \quad (91)$$

Em uma realização lógica exata do modelo quantizado, os dois termos intermediários podem ser zero; o erro em relação ao modelo original permanece devido à quantização e ao hardware.

12.3. Parâmetros como memória navegável Um tensor de pesos pode ser consultado por

$$U_{\Theta} |\ell, i, j\rangle |b\rangle = |\ell, i, j\rangle |b \oplus \text{enc}(\theta_{\ell ij})\rangle, \quad (92)$$

onde ℓ identifica a camada. Esse operador é exatamente uma instância do Navigation Encoding. Existem três estratégias principais:

1. **constantes compiladas:** pesos aparecem diretamente na síntese de somadores e multiplicadores;
2. **memória de parâmetros:** pesos são acessados por U_{Θ} ;
3. **block-encoding:** uma matriz W é incorporada como bloco de uma unidade maior, quando as hipóteses de acesso e normalização são satisfeitas Gilyén et al. (2019).

Nenhuma estratégia torna gratuito o carregamento. A escolha depende de reutilização, esparsidade, largura e algoritmo consumidor.

12.4. Compilação de primitivas não reversíveis Uma camada afim quantizada

$$z = Wx + b \quad (93)$$

pode ser realizada por

$$|x\rangle |0\rangle \mapsto |x\rangle |Wx + b\rangle, \quad (94)$$

preservando x . Ativações não injetivas precisam manter informação suficiente:

$$|x\rangle |0\rangle |0\rangle \mapsto |x\rangle |[x < 0]\rangle |\text{ReLU}(x)\rangle, \quad (95)$$

$$|x_1, \dots, x_k\rangle |0\rangle |0\rangle \mapsto |x_1, \dots, x_k\rangle |\arg \max_i x_i\rangle |\max_i x_i\rangle. \quad (96)$$

Softmax pode ser aproximada por aritmética finita, mas para classificação o argmax dos logits geralmente preserva a classe e evita exponenciais e divisões. Dropout é removido no modo de inferência. Redes feedforward reversibilizadas por ancillas já foram estudadas como base para generalizações quânticas Wan et al. (2017); aqui o foco é a compilação semântica de um modelo clássico fixo.

12.5. Teorema de realização reversível de modelos finitos

Teorema 12.1 (Compilação reversível de inferência). *Seja M um DAG finito e determinístico cujos nós computam funções sobre conjuntos finitos. Suponha que cada operação primitiva ϕ_v admite um circuito reversível que calcula sua saída preservando as entradas. Então existe um circuito unitário limpo U_M tal que*

$$U_M |x\rangle |y\rangle |0^a\rangle = |x\rangle |y \oplus \widehat{f}_{\Theta, p}(x)\rangle |0^a\rangle. \quad (97)$$

Demonstração. Ordene os nós do DAG topologicamente. Compondo os circuitos reversíveis das primitivas, constrói-se A_M que calcula todos os valores intermediários e a saída:

$$A_M |x\rangle |0\rangle = |x\rangle |\widehat{f}(x)\rangle |g(x)\rangle. \quad (98)$$

Copie a saída por XOR para y e aplique $A_M^\dagger$. Pelo procedimento da Equação (31), todos os valores intermediários retornam a zero e resta exatamente (97). Composição e adjunto de unidades são unitários. $\square$

Corolário 12.2 (Existência não implica eficiência). *Se o custo da primitiva v é g_v , uma compilação limpa ingênua possui custo de portas limitado por*

$$G(U_M) \leq 2 \sum_{v \in G_M} g_v + G_{\text{copy}}. \quad (99)$$

Esse limite pode ser impraticável para modelos grandes. Estratégias de pebbling reversível trocam ancillas por recomputação; esparsidade, quantização e fusão de operações podem reduzir custos, mas exigem análise própria.

12.6. Exemplo entendível: perceptron clássico compilado exatamente Considere um perceptron treinado para a função OR em entradas binárias:

$$s = x_1 + x_2, \quad f(x_1, x_2) = \mathbf{1}[s \geq 1]. \quad (100)$$

Uma escolha de pesos é $w = (1, 1)$, bias 0 e limiar 1. A tabela é

x_1	x_2	$f(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	1

Como

$$x_1 \vee x_2 = x_1 \oplus x_2 \oplus (x_1 x_2), \quad (101)$$

o oráculo limpo é realizado por duas CNOTs e uma Toffoli:

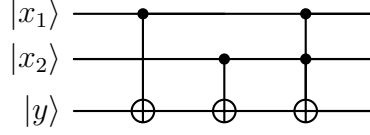


Figura 7. Oráculo reversível do perceptron OR: $y \leftarrow y \oplus f(x_1, x_2)$.

Em superposição uniforme,

$$U_M \left[\frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle) |0\rangle \right] \quad (102)$$

$$= \frac{1}{2} (|00\rangle |0\rangle + |01\rangle |1\rangle + |10\rangle |1\rangle + |11\rangle |1\rangle). \quad (103)$$

O modelo treinado passa a ser um componente coerente. Pode-se marcar a fase das entradas classificadas como positivas, desfazer a inferência e aplicar amplificação de amplitude para pesquisar entradas com uma classe desejada. Esse é o sentido operacional forte de “passar um modelo treinado do mundo clássico para o mundo quântico”.

12.7. O que não é garantido A formalização não implica automaticamente:

- vantagem quântica ou inferência mais rápida;
- circuito pequeno para milhões de parâmetros;
- carregamento de pesos em tempo sublinear;
- preservação exata de ponto flutuante arbitrário;
- executabilidade NISQ de redes profundas;
- equivalência entre copiar pesos para rotações e preservar a função aprendida.

A utilidade quântica aparece quando U_M é chamado coerentemente dentro de um algoritmo que explora superposição, interferência, estimação de amplitude ou busca. Executar uma única inferência reversível pode ser mais caro do que executar o modelo clássico.

13. Exemplo integrado da tese geral Considere uma árvore de decisão simples treinada:

- nó p_0 : testa x_1 ; se 0, segue **left**; se 1, segue **right**;
- folhas p_1 e p_2 : retornam classes 0 e 1.

A estrutura clássica é um grafo tipado com campos **left**, **right**, **feature**, **threshold** e **class**. A Etapa 1 produz N_S com ponteiros canonicalizados e operações **follow** e **read**. A Etapa 2 pode produzir:

$$U_{\text{follow}} |p\rangle |d\rangle |q\rangle = |p\rangle |d\rangle |q \oplus \rho_S(p, d)\rangle, \quad (104)$$

$$U_{\text{class}} |p\rangle |y\rangle = |p\rangle |y \oplus \text{class}(p)\rangle. \quad (105)$$

A inferência completa compõe consulta de feature, comparação reversível, navegação, leitura da classe e uncompute. O grafo lógico possui interações frequentes entre o ponteiro atual e registradores de campo; a realização topology-aware posiciona esses controles em regiões centrais do coupling map. Assim, o mesmo exemplo conecta:

$$\begin{aligned} \text{modelo treinado} &\rightarrow \text{estrutura navegável} \rightarrow \text{álgebra de operações} \\ &\rightarrow \text{unidades} \rightarrow \text{circuito físico.} \end{aligned} \quad (106)$$

14. Propriedades, limites e agenda de prova

14.1. Afirmações já sustentadas

A primeira implementação sustenta:

1. $D \mapsto U_D$ é uma construção unitária e involutiva;
2. a consulta se estende linearmente a endereços em superposição;
3. query–predicate–unquery produz oráculos limpos;
4. o custo lógico inicial depende de $\sum_a h(D[a])$;
5. encoders com menos portas podem realizar tarefas diferentes e não oferecer consulta coerente;
6. a semântica atual é de QRAM lógica/circuital, não de QRAM física escalável.

14.2. Resultados formalizados neste artigo

A generalização estabelece:

1. uma estrutura tipada é tratada até isomorfismo de endereços;
2. a Etapa 1 produz uma IR total e independente de hardware;
3. cada resolvidor finito admite um embedding XOR unitário;
4. a correção da Etapa 2 é expressa por um diagrama comutativo;
5. placement e routing podem ser otimizados sem alterar a semântica;
6. um DAG clássico finito e quantizado admite um oráculo reversível limpo.

14.3. Questões ainda experimentais

Restam demonstrar empiricamente:

- redução real de SWAPs ao escolher representação e placement conjuntamente;
- heurísticas de canonicalização e codificação que superem encode-first-route-later;
- custo de famílias estruturais: listas, árvores, grafos esparsos e modelos treinados;
- robustez sob ruído realista e hardware físico;
- técnicas de pebbling e gerenciamento de ancillas para modelos grandes;
- quando aproximação espectral preserva a tarefa final com menor custo;
- condições sob as quais U_M produz uma vantagem algorítmica após contabilizar carregamento e medição.

15. Discussão A máquina de Turing quântica ocupa um papel fundacional, não decorativo. Ela fornece o espaço de configurações, a linearidade e a exigência de evolução unitária que transformam uma “estrutura clássica” em um problema de representação operatorial. O circuito é a realização finita e composicional dessa dinâmica. A construção U_f é a ponte de reversibilidade; U_D é sua primeira aplicação a uma memória; $U_{\rho,S}$ generaliza a operação para ponteiros e campos; U_M eleva o mesmo princípio à inferência de modelos clássicos.

A divisão em duas etapas evita quatro confusões frequentes:

1. estrutura não é sinônimo de estado quântico;
2. endereço lógico não é sinônimo de qubit físico;
3. operador lógico não é sinônimo de circuito já roteado;
4. transferir pesos não é sinônimo de preservar a função de inferência.

Ela também permite auditoria. A IR estrutural pode ser testada sem Qiskit; a unidade lógica pode ser comparada matricialmente; o circuito roteado pode ser verificado contra a unidade; e o backend pode ser avaliado por erro e custo. Essa separação é compatível com uma arquitetura de middleware quântico, na qual backends diferentes realizam a mesma semântica.

A principal limitação é econômica: reversibilidade garante existência, não eficiência. Preparação arbitrária de estados pode ser exponencial Möttönen et al. (2005); Shende et al. (2006); Plesch and Brukner (2011); QRAM física possui exigências arquiteturais próprias; block-encodings pressupõem modelos de acesso; e compilação de modelos grandes pode duplicar custo por compute–uncompute. Por isso, toda afirmação de vantagem deve contabilizar preparação, consulta, ancillas, routing, ruído e readout Aaronson (2015); Preskill (2018).

16. Conclusão Este artigo deduziu o Navigation Encoding em duas etapas a partir dos fundamentos físicos e computacionais. Espaços de Hilbert descrevem estados; operadores unitários descrevem evolução coerente; máquinas de Turing quânticas justificam a unitariedade global; circuitos fornecem uma realização finita; computação reversível permite incorporar funções clássicas sem apagar informação.

A unidade de memória

$$U_D |a\rangle |b\rangle = |a\rangle |b \oplus D[a]\rangle \quad (107)$$

é o caso particular inicial. A tese geral é

$$\boxed{\text{NavEnc}_G = \text{Realize}_G \circ \text{StructEnc}} \quad (108)$$

com preservação semântica, topologia física explícita e erro controlado. A primeira etapa recebe uma estrutura ou modelo em sua linguagem natural; a segunda transforma suas operações em unidades, seleciona codificação, ancillas, placement, routing e síntese.

Como resultado final, um modelo clássico treinado de precisão finita pode ser compilado para

$$U_M |x\rangle |y\rangle |0^a\rangle = |x\rangle |y \oplus \hat{f}_{\Theta,p}(x)\rangle |0^a\rangle. \quad (109)$$

Esse resultado torna a função aprendida utilizável coerentemente em algoritmos quânticos, sem prometer eficiência universal. A próxima etapa científica é implementar o pipeline completo, provar preservação semântica em IRs concretas e medir quando a otimização conjunta de estrutura e topologia reduz custo físico real.

Referências

- Aaronson, S. (2015). Read the fine print. *Nature Physics*, 11:291–293.
- Bennett, C. H. (1973). Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532.
- Bény, C. and Richter, F. (2015). Algebraic approach to quantum theory: A finite-dimensional guide. *arXiv preprint arXiv:1505.03106*.
- Bernstein, E. and Vazirani, U. (1997). Quantum complexity theory. *SIAM Journal on Computing*, 26(5):1411–1473.
- Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., and Lloyd, S. (2017). Quantum machine learning. *Nature*, 549:195–202.
- Chung, F. R. K. (1997). *Spectral Graph Theory*. American Mathematical Society.
- Cowtan, A., Dilkes, S., Duncan, R., Simmons, W., and Sivarajah, S. (2019). On the qubit routing problem. *Theory of Computing Systems*, 63:1672–1717.
- Deutsch, D. (1985). Quantum theory, the church–turing principle and the universal quantum computer. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, 400(1818):97–117.
- Duncan, R., Kissinger, A., Perdrix, S., and van de Wetering, J. (2020). Graph-theoretic simplification of quantum circuits with the zx-calculus. *Quantum*, 4:279.
- Feitosa, J. Y. d. O. (2026a). Encoding quântico por navegação coerente em estruturas hierárquicas de dados para quantum machine learning. Technical report, Universidade Federal Rural de Pernambuco, Unidade Acadêmica de Belo Jardim. Relatório técnico-científico não publicado.
- Feitosa, J. Y. d. O. (2026b). Revisão bibliográfica matemática: Espaços, estruturas algébricas e operadores unitários para modelagem de qubits e navigation encoding. Technical report, Universidade Federal Rural de Pernambuco. Relatório de revisão não publicado.
- Gilyén, A., Su, Y., Low, G. H., and Wiebe, N. (2019). Quantum singular value transformation and beyond: Exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, pages 193–204.
- Giovannetti, V., Lloyd, S., and Maccone, L. (2008a). Architectures for a quantum random access memory. *Physical Review A*, 78:052310.

- Giovannetti, V., Lloyd, S., and Maccone, L. (2008b). Quantum random access memory. *Physical Review Letters*, 100:160501.
- Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219.
- Havlíček, V., Córcoles, A. D., Temme, K., Harrow, A. W., Kandala, A., Chow, J. M., and Gambetta, J. M. (2019). Supervised learning with quantum-enhanced feature spaces. *Nature*, 567:209–212.
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713.
- Landsman, N. P. (1998). Lecture notes on c^* -algebras, hilbert c^* -modules, and quantum mechanics. *arXiv preprint math-ph/9807030*.
- Li, G., Ding, Y., and Xie, Y. (2019). Tackling the qubit mapping problem for nisq-era quantum devices. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 1001–1014.
- Möttönen, M., Vartiainen, J. J., Bergholm, V., and Salomaa, M. M. (2005). Transformation of quantum states using uniformly controlled rotations. *Quantum Information and Computation*, 5(6):467–473.
- Nielsen, M. A. and Chuang, I. L. (2010). *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition.
- Ozawa, M. and Nishimura, H. (2000). Local transition functions of quantum turing machines. *RAIRO - Theoretical Informatics and Applications*, 34(5):379–402.
- Plesch, M. and Brukner, Č. (2011). Quantum-state preparation with universal gate decompositions. *Physical Review A*, 83:032302.
- Preskill, J. (2018). Quantum computing in the nisq era and beyond. *Quantum*, 2:79.
- Schuld, M. and Killoran, N. (2019). Quantum machine learning in feature hilbert spaces. *Physical Review Letters*, 122(4):040504.
- Shende, V. V., Bullock, S. S., and Markov, I. L. (2006). Synthesis of quantum-logic circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 25(6):1000–1010.
- Siraichi, M. Y., Santos, V. F. d., Collange, S., and Pereira, F. M. Q. (2018). Qubit allocation. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization*, pages 113–125.
- Spielman, D. A. and Srivastava, N. (2011). Graph sparsification by effective resistances. *SIAM Journal on Computing*, 40(6):1913–1926.

- Wan, K. H., Dahlsten, O., Kristjánsson, H., Gardner, R., and Kim, M. S. (2017). Quantum generalisation of feedforward neural networks. *npj Quantum Information*, 3:36.
- Watrous, J. (2018). *The Theory of Quantum Information*. Cambridge University Press.
- Yao, A. C.-C. (1993). Quantum circuit complexity. In *Proceedings of the 34th Annual Symposium on Foundations of Computer Science*, pages 352–361. IEEE.